

# AI in the Real World: Trade-offs Behind Fast & Scalable Object Detection

*How to build vision systems that run on multiple cameras — without skipping a frame*



**Teerapong Panboonyuen, Ph.D. (P'Kao)**

Super AI Engineer Thailand 2026 | SAIE Workshop | 3 Hours · 6 Labs · Google Colab T4

Object Detection

Model Optimization

Edge AI

VisDrone

Get to know me at <https://kaopanboonyuen.github.io/>

# Workshop Roadmap — 6 Labs, 3 Hours

## Lab 1

### Baseline Profiling

FLOPs · Latency · mAP

---

~30 min | [Google Colab](#)

## Lab 2

### Structured Pruning

Channel Pruning + Fine-tune

---

~30 min | [Google Colab](#)

## Lab 3

### Quantization

FP32 → FP16 → INT8

---

~30 min | [Google Colab](#)

## Lab 4

### Knowledge Distillation

Teacher → Student

---

~30 min | [Google Colab](#)

## Lab 5

### Multi-Scale Heads

FPN · DW-Sep · NAS Concepts

---

~30 min | [Google Colab](#)

## Lab 6

### Deployment & Scale

ONNX · Multi-Camera · Edge

---

~30 min | [Google Colab](#)

# Why Does This Matter? — The Real-World Problem

## The Challenge from Industry

A real company needs object detection that can:

- Run fast — low latency per frame
- Handle many cameras simultaneously — without lag
- Run on embedded boards like Jetson Nano — minimal power
- Keep accuracy high enough to be trustworthy

4 Cameras × 25 FPS

= 100 FPS total  
needed  
from a single GPU

Embedded Hardware

Jetson Nano: only  
4GB RAM, 128 CUDA  
cores

mAP Must Hold

Accuracy  $\geq$  80%  
of the baseline model

## The Engineering Philosophy Behind This Workshop

**Every optimization must be measurable.**

We never say "it's faster" without showing numbers.

- We track: mAP@50, FLOPs, Latency (ms), Throughput (FPS), Parameters (M)

# Dataset — VisDrone (SAIE 2026 Subset)

[Setup](#)

## What is VisDrone?

VisDrone is a large-scale benchmark dataset captured by drone cameras at varying altitudes, lighting, and weather conditions.

### 10 object classes for urban traffic monitoring:

- pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, motor
- **Key challenge: Severe class imbalance (pedestrian >> bus)**
- **Objects are tiny — many < 32×32 pixels in drone footage**

~800

Train Images

~200

Val Images

10

Object Classes

416×416

Input Size

5,000

Pretrained Epochs

YOLOv8s

Model

## Why VisDrone for This Workshop?

Tiny objects from drone altitude stress-test every optimization. A pruned model that loses accuracy on pedestrians can have real safety consequences — this dataset keeps us honest.

## Sample Data and Annotations

The VisDrone dataset contains a diverse set of images and videos captured by drone-mounted cameras. Here are some examples of data from the dataset, along with their corresponding annotations:



```
CLASS_NAMES = {  
    0: "pedestrian",  
    1: "people",  
    2: "bicycle",  
    3: "car",  
    4: "van",  
    5: "truck",  
    6: "tricycle",  
    7: "awning-tricycle",  
    8: "bus",  
    9: "motor"  
}
```

- **Task 1: Object detection** in images – This image demonstrates an example of object detection in images, where objects are annotated with **bounding boxes**. The dataset provides a wide variety of images taken from different locations, environments, and densities to facilitate the development of models for this task.

## What are the main subsets of the VisDrone dataset and their applications?

The VisDrone dataset is divided into five main subsets, each tailored for a specific computer vision task:

1. **Task 1:** Object detection in images.
2. **Task 2:** Object detection in videos.
3. **Task 3:** Single-object tracking.
4. **Task 4:** Multi-object tracking.
5. **Task 5:** Crowd counting.

These subsets are widely used for training and evaluating [deep learning](#) models in drone-based applications such as surveillance, traffic monitoring, and public safety.

# Key Metrics We Track — Definitions & Equations

## mAP@50

Mean Average Precision at IoU=0.5

$$\text{mAP} = (1/|C|) \times \sum \text{AP}_c$$

Average of per-class Average Precision across all C classes.  
Primary accuracy metric for object detection.

## mAP@50-95

Strict mAP (averaged over IoU 0.5→0.95)

$$\text{mAP}_{50:95} = (1/10) \times \sum \text{mAP}_{\text{IoU}}$$

Averages over 10 IoU thresholds. Penalizes imprecise boxes. Used in COCO benchmark.

## FLOPs

Floating Point Operations

$$\text{FLOPs} = 2 \times C_{\text{in}} \times k^2 \times C_{\text{out}} \times H \times W$$

Total multiply-adds in a forward pass. Measures computational cost. Lower = faster model.

## Latency

Inference Time per Image (ms)

$$\text{Latency} = t_{\text{end}} - t_{\text{start}} \text{ (ms)}$$

Wall-clock time for one inference. Measure after GPU warm-up! Use P95 for worst-case.

## FPS

Frames Per Second (Throughput)

$$\text{FPS} = 1000 / \text{mean\_latency\_ms}$$

How many frames per second the model can process.  
Must be  $\geq 25$  FPS per camera for real-time.

## Efficiency

Accuracy-per-Compute Trade-off

$$\text{Efficiency} = \text{mAP}_{50} / \text{GFLOPs}$$

Measures how much accuracy you get per unit of compute. Higher = better. Helps compare models fairly.

# YOLOv12: Attention-Centric Real-Time Object Detectors

**Yunjie Tian**  
University at Buffalo  
yunjieti@buffalo.edu

**Qixiang Ye\***  
UCAS  
qxyc@ucas.ac.cn

**David Doermann**  
University at Buffalo  
doermann@buffalo.edu

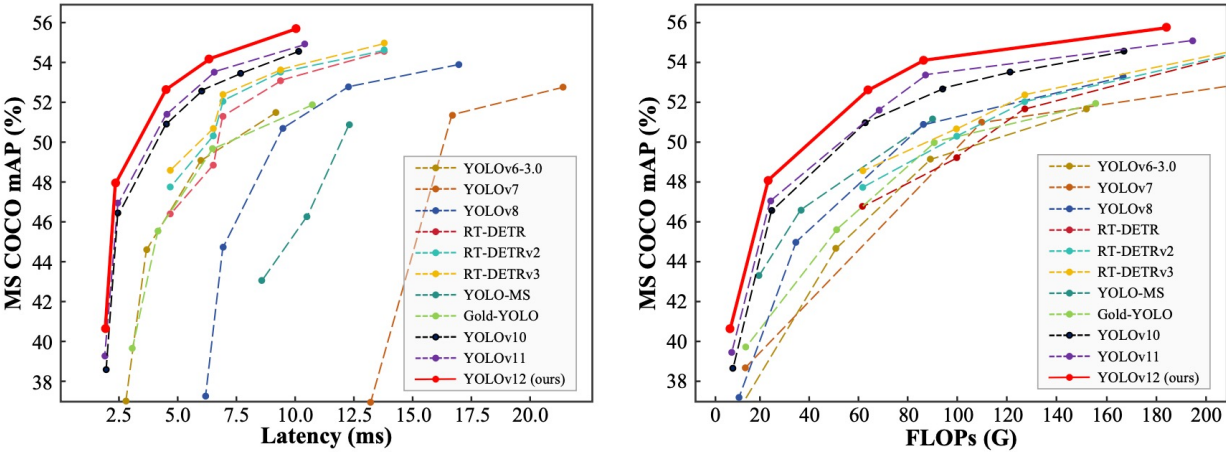
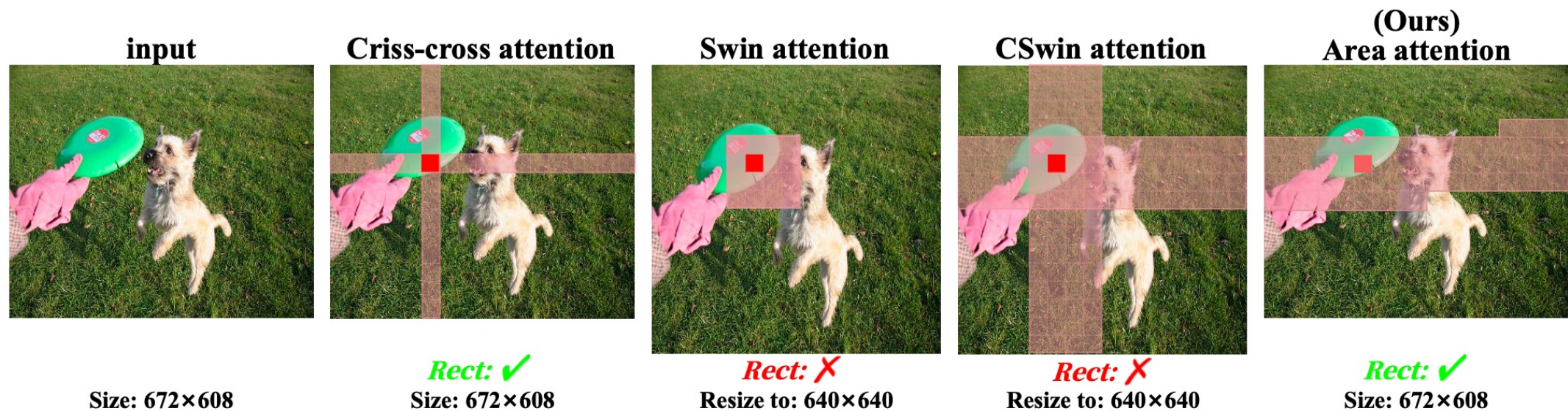


Figure 1: Comparisons with other popular methods in terms of latency-accuracy (left) and FLOPs-accuracy (right) trade-offs.

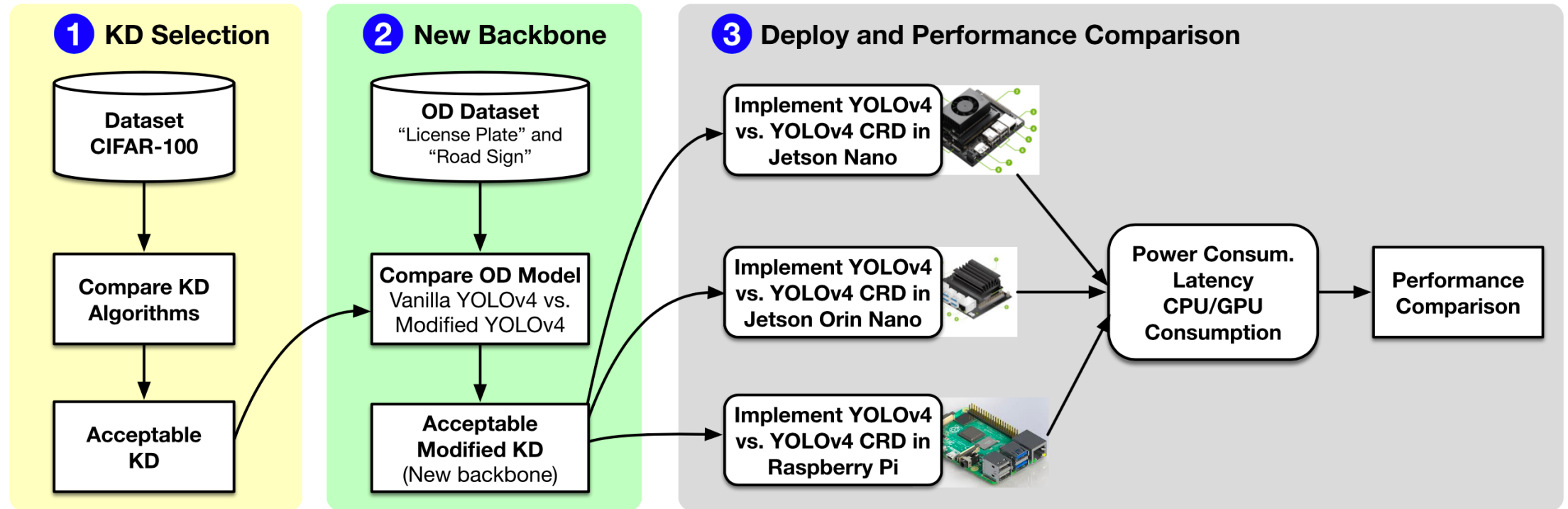




**Figure 2: Comparison of the representative local attention mechanisms with our area attention.** Area attention computes local attention over sequentially placed tokens without requiring input size constraints, enabling default distortion-free rectangular inference (*Rect*) in YOLO. Area attention enjoys efficient implementation, requiring only flatten and a reshape operations.

Table 1: **Comparison with popular real-time object detectors.** All results are obtained using  $640 \times 640$  inputs. †: pre-trained models are required.

| Method                  | FLOPs<br>(G) | #Param.<br>(M) | $AP_{50:95}^{val}$<br>(%) | $AP_{50}^{val}$<br>(%) | $AP_{75}^{val}$<br>(%) | Latency<br>(ms) |
|-------------------------|--------------|----------------|---------------------------|------------------------|------------------------|-----------------|
| YOLOv8-N [26]           | 8.7          | 3.2            | 37.4                      | 52.6                   | 40.5                   | 1.77            |
| YOLOv10-N [61]          | 6.7          | 2.3            | 38.5                      | 53.8                   | 41.7                   | 1.84            |
| YOLO11-N [31]           | 6.5          | 2.6            | 39.4                      | 55.3                   | 42.8                   | 1.5             |
| <b>YOLOv12-N (Ours)</b> | <b>6.0</b>   | <b>2.6</b>     | <b>40.5</b>               | <b>56.4</b>            | <b>43.8</b>            | <b>1.62</b>     |
| YOLOv8-S [26]           | 28.6         | 11.2           | 45.0                      | 61.8                   | 48.7                   | 2.33            |
| RT-DETRv2-R18† [44]     | 60.0         | 20.0           | 47.9                      | 64.9                   | —                      | 4.58            |
| YOLOv9-S [67]           | 26.4         | 7.1            | 46.8                      | 63.4                   | 50.7                   | —               |
| YOLOv10-S [61]          | 21.6         | 7.2            | 46.3                      | 63.0                   | 50.4                   | 2.49            |
| YOLO11-S [31]           | 21.5         | 9.4            | 46.9                      | 63.9                   | 50.6                   | 2.5             |
| <b>YOLOv12-S (Ours)</b> | <b>19.5</b>  | <b>9.1</b>     | <b>47.8</b>               | <b>64.9</b>            | <b>51.3</b>            | <b>2.44</b>     |
| YOLOv8-M [26]           | 78.9         | 25.9           | 50.3                      | 67.2                   | 54.7                   | 5.09            |
| RT-DETRv2-R34† [44]     | 100.0        | 36.0           | 49.9                      | 67.5                   | —                      | 6.32            |
| RT-DETRv3-R18† [44]     | 60.0         | 20.0           | 48.7                      | —                      | —                      | 4.58            |
| RT-DETRv3-R34† [69]     | 100.0        | 36.0           | 50.1                      | 67.5                   | —                      | 6.32            |
| YOLOv9-M [67]           | 76.3         | 20.0           | 51.4                      | 68.1                   | 56.1                   | —               |
| YOLOv10-M [61]          | 59.1         | 15.4           | 51.1                      | 68.1                   | 55.8                   | 4.74            |
| YOLO11-M [31]           | 68.0         | 20.1           | 51.5                      | 68.5                   | 55.7                   | 4.7             |
| <b>YOLOv12-M (Ours)</b> | <b>59.9</b>  | <b>19.7</b>    | <b>52.5</b>               | <b>70.0</b>            | <b>57.0</b>            | <b>4.30</b>     |



**FIGURE 1.** Our research pipeline.

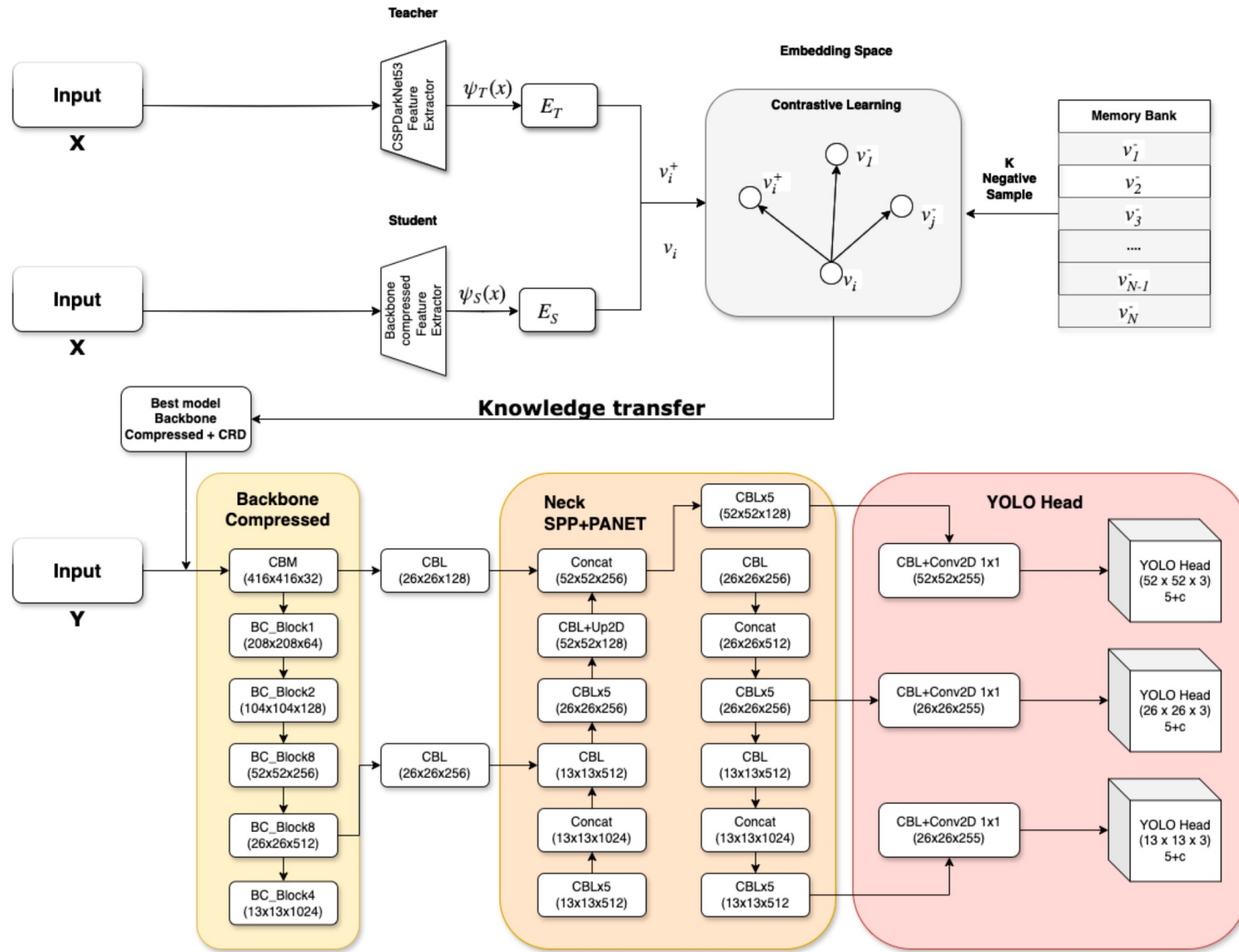
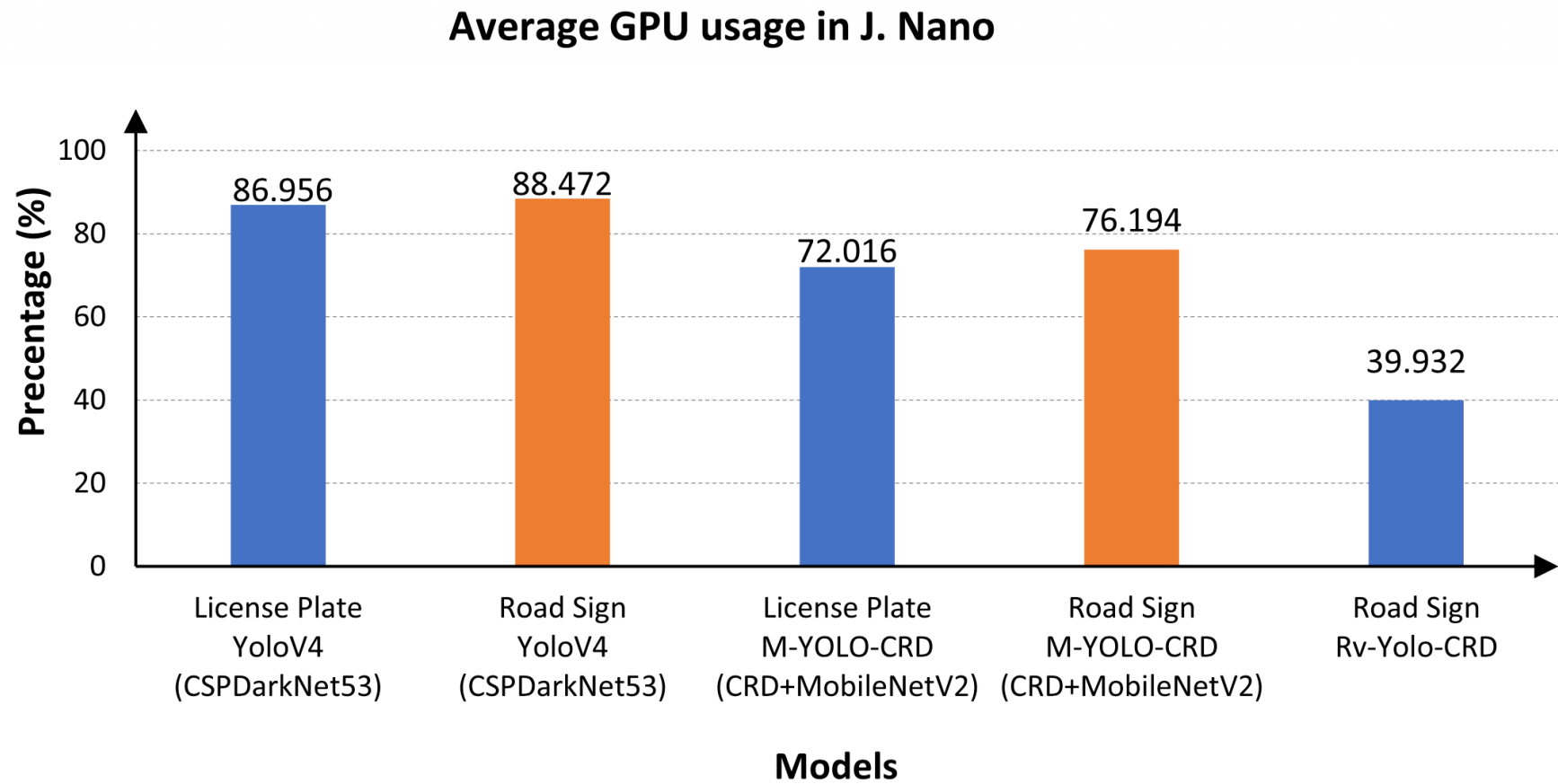


FIGURE 2. Proposed Method: Compressed backbone with CRD and its integration into YOLOv4.





**FIGURE 3.** Average GPU usage in edge devices based GPU.

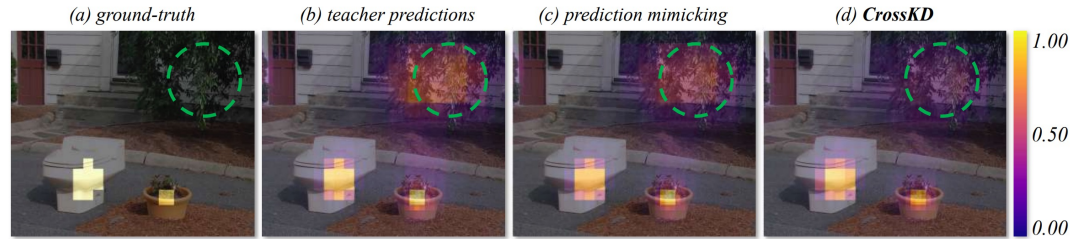


Figure 2. Visualizations of the classification predictions from the GFL [35]. (a) and (b) are ground truth and distillation targets. (c) and (d) are the classification outputs predicted by models training with conventional prediction mimicking and proposed CrossKD. In the green circled areas, the distillation targets predicted by the teacher have a large discrepancy with the ground-truth targets assigned to the student. prediction mimicking forces the student to mimic the teacher, while CrossKD can smooth the mimicking process.

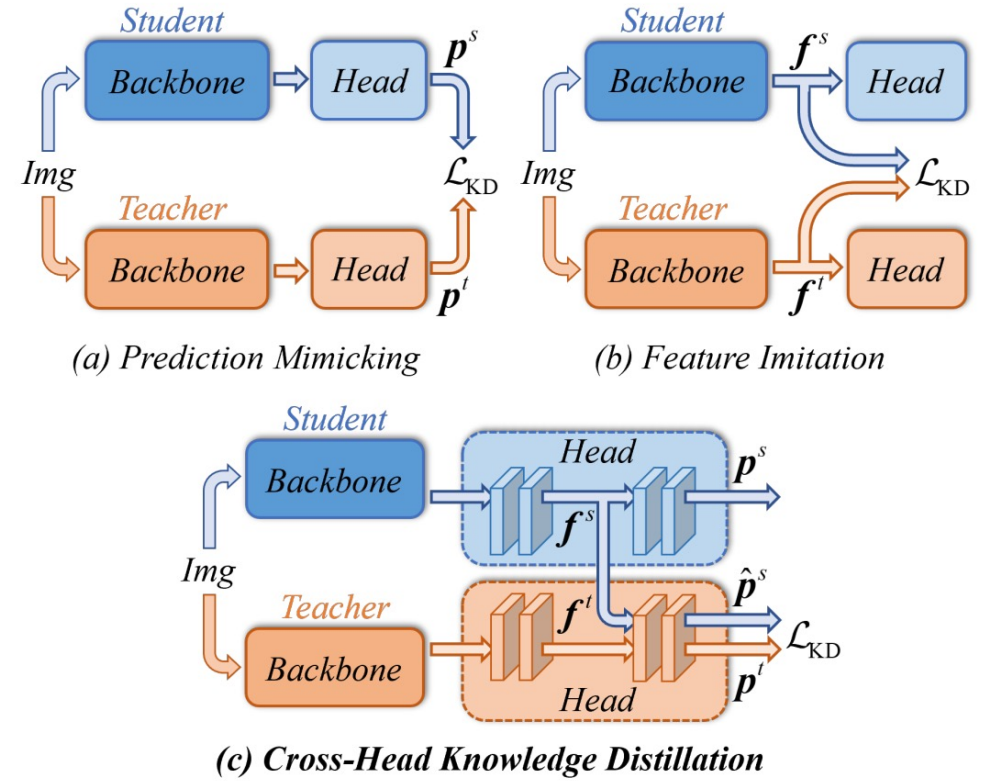


Figure 1. Comparisons between conventional KD methods and our CrossKD. Rather than explicitly enforcing the consistency between the intermediate feature maps or the predictions of the teacher-student pair, CrossKD implicitly builds the connection between the heads of the teacher-student pair to improve the distillation efficiency.

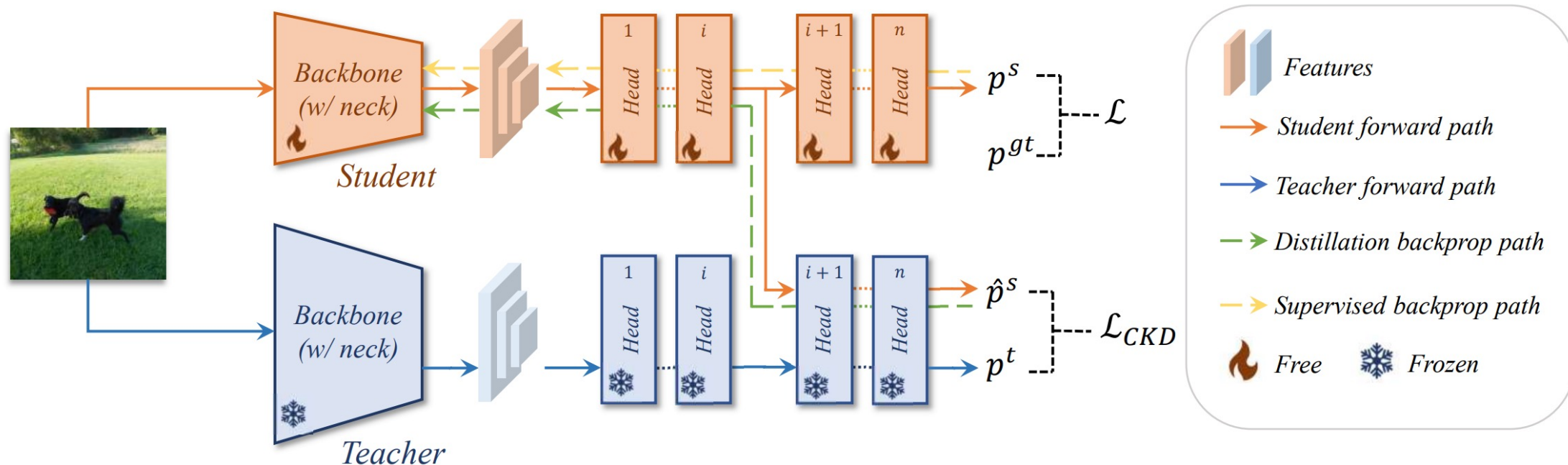


Figure 4. Overall framework of the proposed CrossKD. For a given teacher-student pair, CrossKD first delivers the intermediate features of the student into the teacher layers and generates the cross-head predictions  $\hat{p}^s$ . Then, distillation losses are calculated between the original teacher's predictions and the cross-head predictions of the student. In back-propagation, the gradients with respect to the detection loss normally pass through the student detection head, while the distillation gradients propagate through the frozen teacher layers.

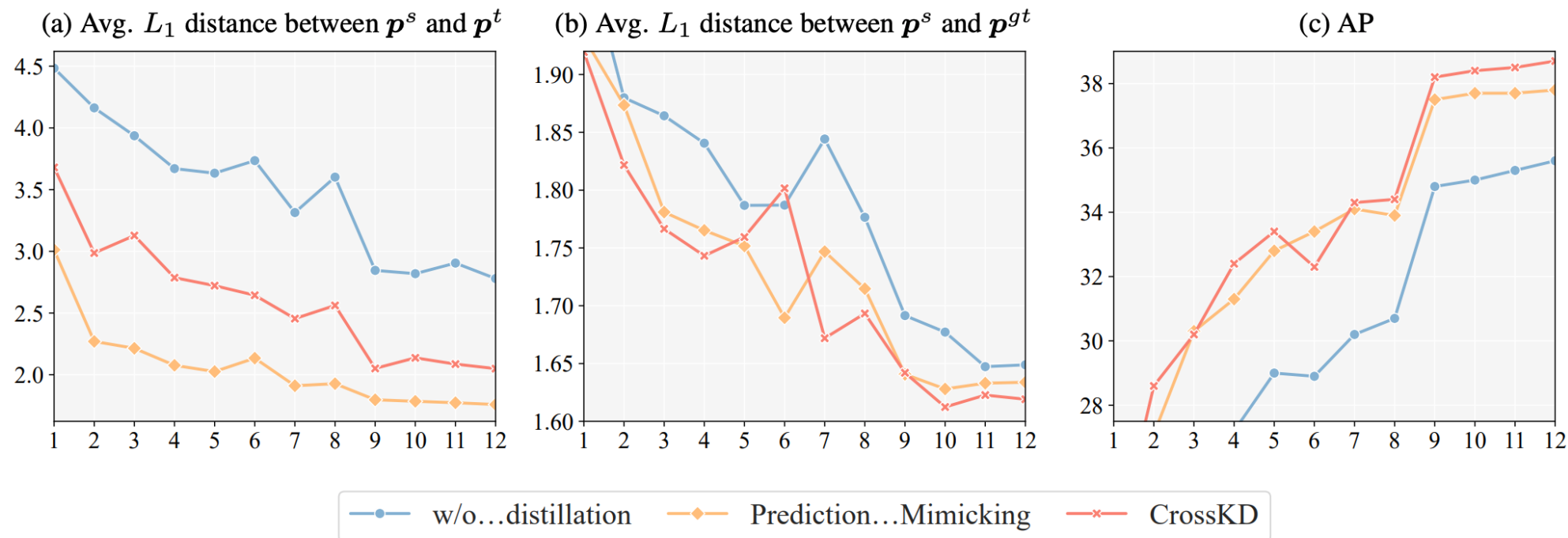
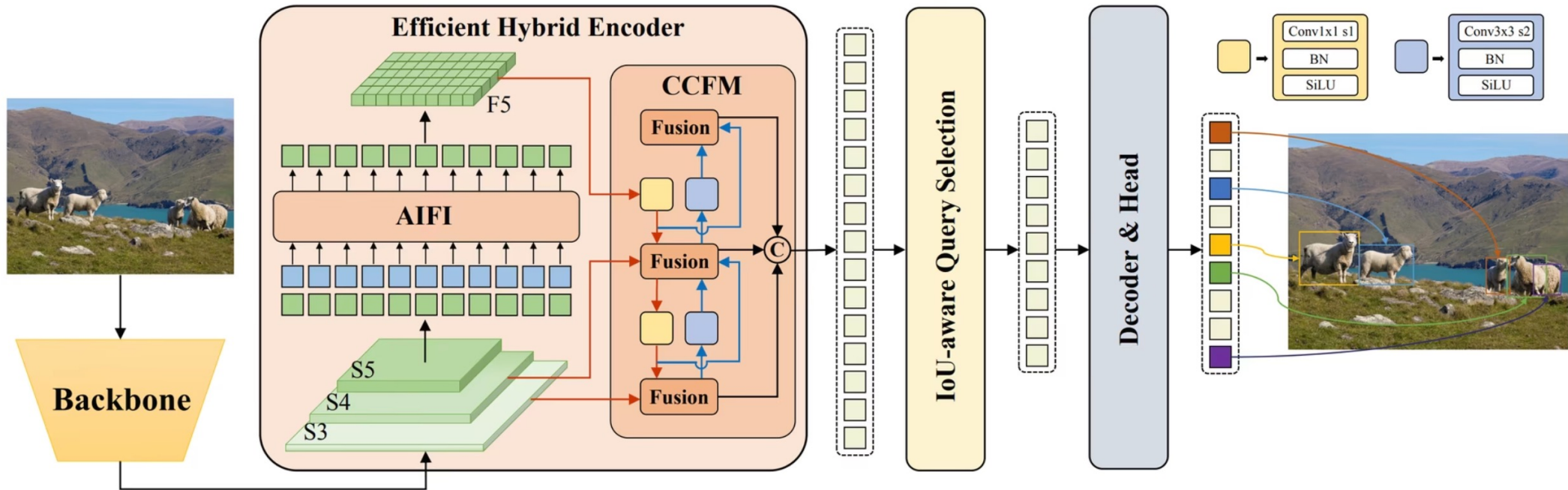


Figure 6. Visualization for the variation of statistics during training. (a) Curves of average  $L_1$  distance between student predictions  $p^s$  and teacher's  $p^t$ . (b) Curves of average  $L_1$  distance between student predictions  $p^s$  and positive ground truth targets  $p^{gt}$ . (c) Curves of Average Precision (AP). All curves are evaluated on the COCO *minival* set. X-axis refers to the epoch number. Y-axis in (a) and (b) indicate the average  $L_1$  distance, while in (c) means the value of AP.





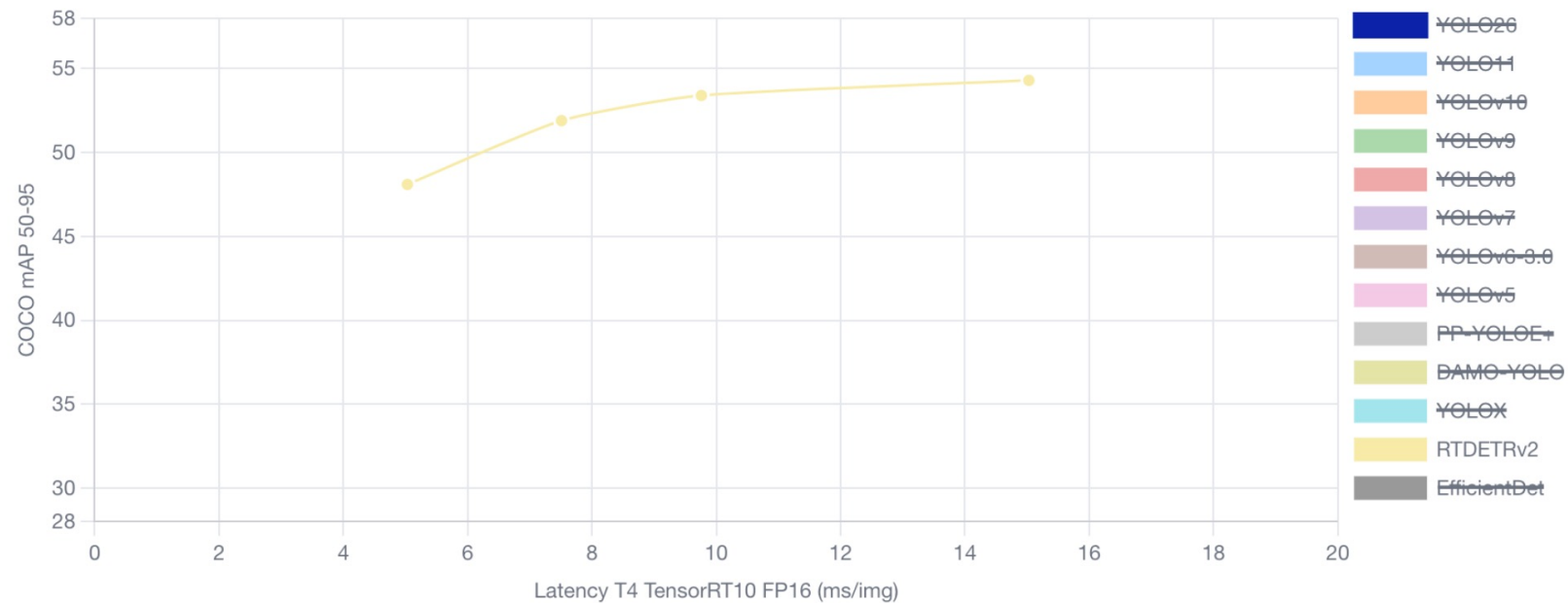
**Overview of Baidu's RT-DETR.** The RT-DETR model architecture diagram shows the last three stages of the backbone {S3, S4, S5} as the input to the encoder. The efficient hybrid encoder transforms multiscale features into a sequence of image features through intrascale feature interaction (AIFI) and cross-scale feature-fusion module (CCFM). The IoU-aware query selection is employed to select a fixed number of image features to serve as initial object queries for the decoder. Finally, the decoder with auxiliary prediction heads iteratively optimizes object queries to generate boxes and confidence scores ([source](#)).

## Pretrained Models

The Ultralytics Python API provides pretrained PaddlePaddle RT-DETR models with different scales:

- RT-DETR-L: 53.0% AP on COCO val2017, 114 FPS on T4 GPU
- RT-DETR-X: 54.8% AP on COCO val2017, 74 FPS on T4 GPU

Additionally, Baidu has released RTDETRv2 in July 2024, which further improves upon the original architecture with enhanced performance metrics.



## Example

Python

CLI

```
from ultralytics import RTDETR

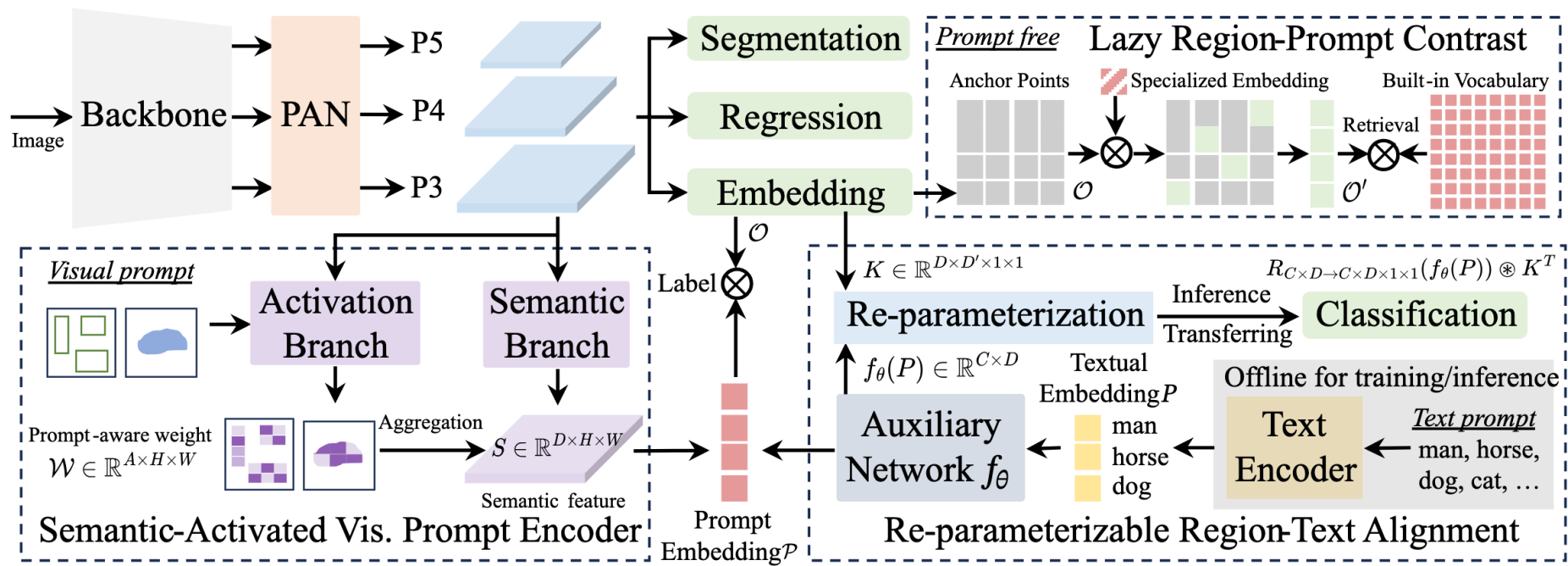
# Load a COCO-pretrained RT-DETR-1 model
model = RTDETR("rtdetr-1.pt")

# Display model information (optional)
model.info()

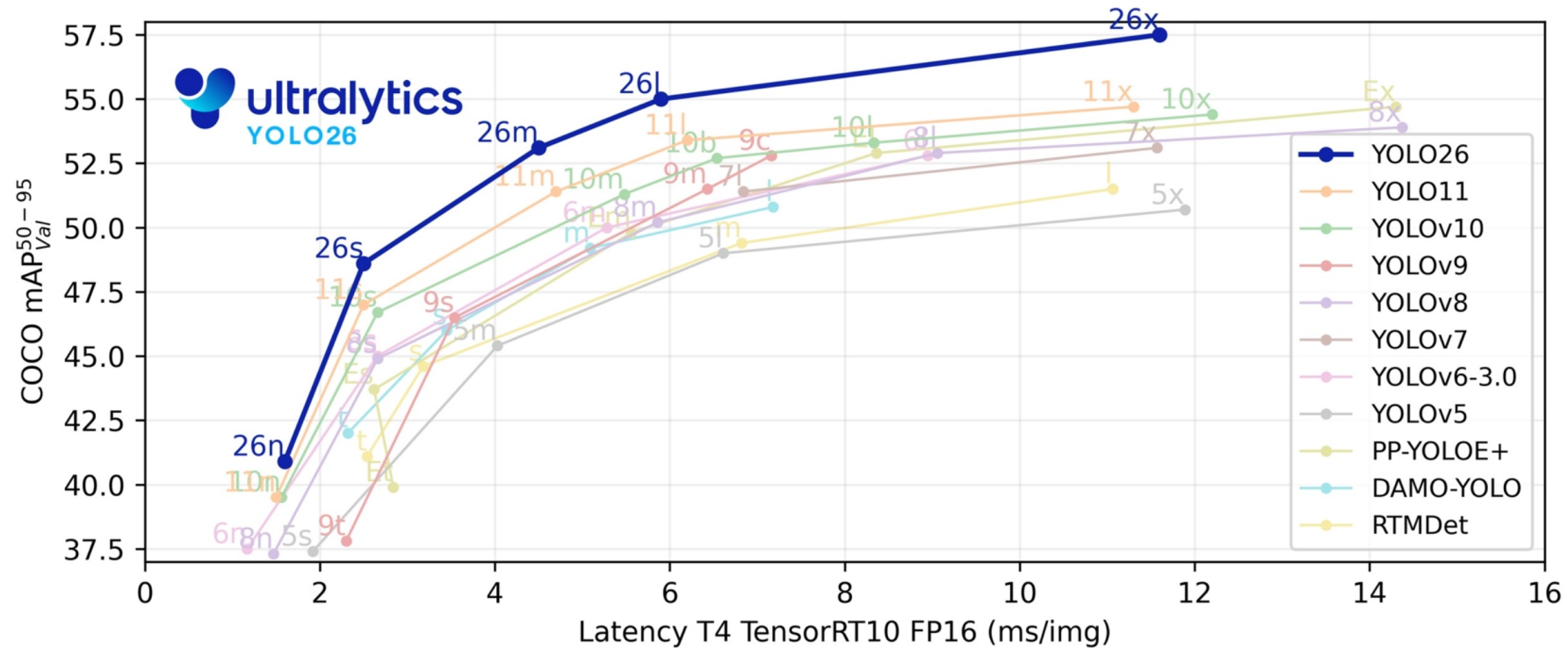
# Train the model on the COCO8 example dataset for 100 epochs
results = model.train(data="coco8.yaml", epochs=100, imgsz=640)

# Run inference with the RT-DETR-1 model on the 'bus.jpg' image
results = model("path/to/bus.jpg")
```

# Architecture Overview

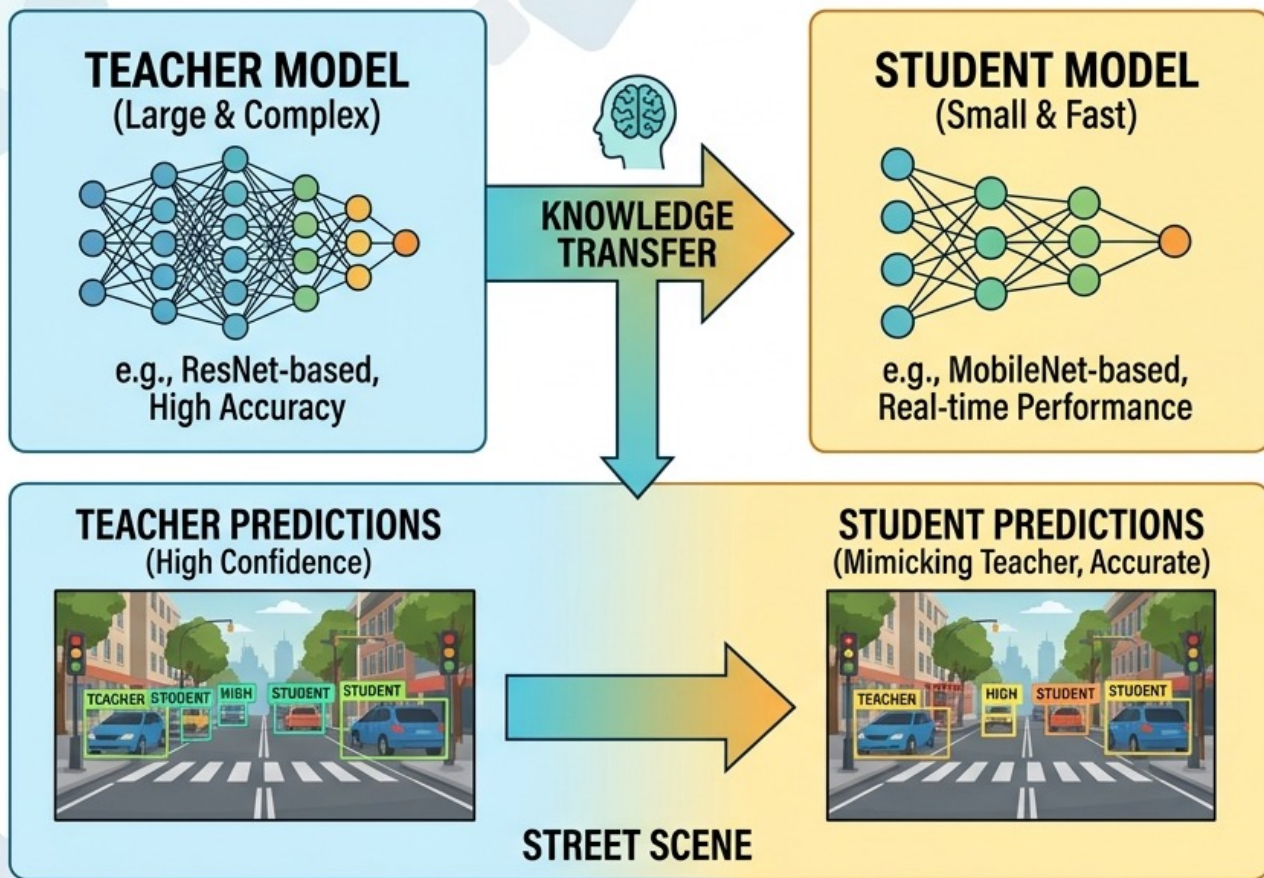


YOLOE retains the standard YOLO structure—a convolutional **backbone** (e.g., CSP-Darknet) for feature extraction, a **neck** (e.g., PAN-FPN) for multi-scale fusion, and an **anchor-free, decoupled** detection **head** (as in YOLOv8/YOLO11) predicting objectness, classes, and boxes independently. YOLOE introduces three novel modules enabling open-vocabulary detection:



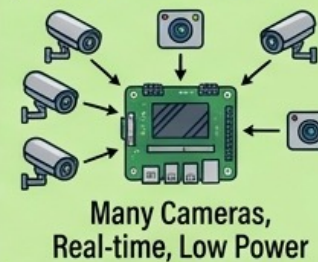


# BASIC MODEL DISTILLATION FOR OBJECT DETECTION: **TEACHING AI EFFICIENCY**



## REAL-WORLD APPLICATION: THE TRADE-OFF

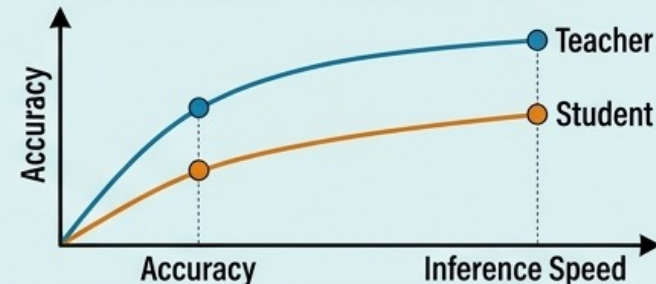
### SCENARIO 1: FAST & SCALABLE (Student Model on Jetson Nano)



### SCENARIO 2: HIGH ACCURACY (Teacher Model on Server)



## PERFORMANCE VS. COMPLEXITY



**KEY TAKEAWAY:** Distillation transfers accuracy from slow, large models to fast, compact ones for efficient, real-world object detection deployment.



Lab 1

# Baseline Profiling & Benchmarking

---

*Objective: Measure everything BEFORE we optimize.  
You cannot improve what you don't measure.*

→ Open Google Colab to follow along

# Lab 1 — How mAP is Computed (Step by Step)

## Average Precision (AP) — The Building Block

- Step 1: For each class  $c$ , rank all detections by confidence score (highest first)
- Step 2: At each rank  $k$ , compute Precision  $P(k)$  and Recall  $R(k)$
- Step 3: Integrate the Precision-Recall curve  $\rightarrow$  Area =  $AP_c$
- Step 4: Average  $AP_c$  across all  $C$  classes  $\rightarrow$  mAP

### Key Equations

$$\text{Precision}(k) = TP(k) / (TP(k) + FP(k))$$

$$\text{Recall}(k) = TP(k) / \text{Total Ground Truth}$$

$$AP_c = \int_0^1 P(r) dr \approx \sum_k P(k) \times \Delta R(k)$$

$$mAP = (1/|C|) \times \sum_{\{c \in C\}} AP_c$$

Where  $k$  = rank,  $TP$  = True Positive,  $FP$  = False Positive

### What Do These Numbers Mean in Practice?

**mAP@50 = 0.9+**

Excellent. Near-perfect boxes at IoU $\geq$ 0.5

**mAP@50 = 0.7–0.9**

Good. Suitable for most real-world deployments

**mAP@50 = 0.5–0.7**

Acceptable. May miss small or occluded objects

**mAP@50 < 0.5**

Poor. Missing too many detections

## Mathematical Foundation

$$\text{mAP} = \frac{1}{|C|} \sum_{c \in C} AP_c$$

$$AP_c = \int_0^1 p(r) dr \approx \sum_{k=1}^N P(k) \cdot \Delta R(k)$$

Where  $P(k)$  is precision at rank  $k$ , and  $\Delta R(k)$  is the change in recall.

$$\text{FPS} = \frac{1}{\text{Average Latency per Frame (s)}}$$

$$\text{Efficiency Score} = \frac{\text{mAP}}{\text{GFLOPs}} \quad (\text{higher} = \text{better})$$

## FLOPs — Floating Point Operations

For a single Conv2D layer:

$$\text{FLOPs} = 2 \times C_{\text{in}} \times k^2 \times C_{\text{out}} \times H \times W$$

- $C_{\text{in}}, C_{\text{out}}$  = input/output channels
- $k$  = kernel size (e.g. 3 for 3×3 conv)
- $H \times W$  = feature map spatial dimensions
- The factor 2 counts both multiply and add

### Why FLOPs matter:

Fewer FLOPs → faster inference → more cameras per GPU.  
YOLOv8s ≈ 28 GFLOPs · YOLOv8n ≈ 8 GFLOPs (3.5× less!)

## Latency Measurement — The Right Way

- **Always warm up the GPU first!**

The first few inferences are slow due to CUDA kernel compilation and memory allocation.

- **Use `torch.cuda.synchronize()` after each call**

GPU runs asynchronously — without `synchronize()`, you measure scheduling time, not computation.

- **Measure P50 and P95 (not just mean)**

P95 tells you the worst-case lag a user will experience. For real-time video, P95 latency matters most.

$$\text{FPS} = 1000 / \text{mean\_latency\_ms} \rightarrow \text{Target: } \geq 25 \text{ FPS per camera}$$

# Lab 1 — YOLOv8 Model Family & the Pareto Frontier

*No single model wins everywhere. Always measure the trade-off.*

| Model   | Parameters | GFLOPs | FPS (T4) | mAP@50 (COCO) |
|---------|------------|--------|----------|---------------|
| YOLOv8n | 3.2M       | 8.7G   | 160      | 0.37          |
| YOLOv8s | 11.2M      | 28.6G  | 120      | 0.45          |
| YOLOv8m | 25.9M      | 78.9G  | 80       | 0.50          |
| YOLOv8l | 43.7M      | 165G   | 50       | 0.53          |

## The Pareto Frontier Concept

On the Pareto frontier, you cannot improve one metric without hurting another.

- **YOLOv8n: fastest, lowest accuracy, fits 4+ cameras on Jetson Nano**
- **YOLOv8s: our SAIE baseline — good balance for training demonstration**
- **YOLOv8m/l: high accuracy, too slow for multi-camera real-time on edge hardware**

*Homework — Compute Efficiency = mAP@50 / Params(M) for each model. Which wins?*



Lab 2

# Structured Pruning: Removing What Doesn't Matter

---

*Objective: Cut redundant channels from the backbone → fewer FLOPs, same task.  
Fine-tune to recover lost accuracy.*

→ Open Google Colab to follow along

## What is Pruning?

A neural network has many redundant neurons — channels that barely change the output.

Structured pruning permanently removes entire channels (filters), producing a genuinely smaller and faster model.

- **Unstructured (weight masking) only sets weights to zero — does NOT reduce FLOPs on GPU!**

### Sparsity Regularization During Training

$$L_{\text{total}} = L_{\text{detection}} + \lambda \times \sum |\gamma_i|$$

Adding L1 on  $\gamma$  during training pushes channels to zero  $\rightarrow$  easier to prune cleanly later.

### Channel Importance Score (L1-Norm)

For filter  $i$  in a Conv2D layer:

$$s_i = \|F_i\|_1 = \sum |F_i(j,k,l)|$$

Channels with lowest  $s_i$  contribute least  $\rightarrow$  safe to remove.

### BatchNorm-Based Pruning (Better for YOLOv8)

BatchNorm has a scale factor  $\gamma$  (gamma) that gates channels:

$$y = \gamma \times \hat{x} + \beta \quad \text{where } \hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$$

**If  $|\gamma| \approx 0 \rightarrow$  output is zero  $\rightarrow$  channel is prunable!**

## Lab 2 — Pruning Workflow & What to Expect

Lab 2

| Step 1                                                                                                                                                                | Step 2                                                                                                                                                                             | Step 3                                                                                                                                                  | Step 4                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Analyze BN <math>\gamma</math> Distribution</b><br><br>Plot histogram of $ \gamma $ values across all BatchNorm layers. Channels near zero are pruning candidates. | <b>Rank &amp; Prune Channels</b><br><br>Remove bottom-p% channels by $ \gamma $ score. Typical $p = 20\text{--}40\%$ . More pruning = more FLOPs reduction but more accuracy loss. | <b>Fine-Tune the Model</b><br><br>The model accuracy drops after pruning. Fine-tune for a few epochs with a lower learning rate to recover performance. | <b>Measure: <math>\Delta</math>Params, <math>\Delta</math>FLOPs, <math>\Delta</math>mAP</b><br><br>Always quantify the trade-off. Report how much speed you gained and how much accuracy you lost. |

### Expected Results at 30% Pruning Ratio

**~20–40%**

FLOPs Reduction

**1.2–2×**

Latency Improvement

**~2–5%**

mAP Drop

**Medium**

Effort

Lab 3

# Post-Training Quantization: FP32 → FP16 → INT8

---

*Objective: Shrink model size and speed up inference by using fewer bits per weight — without retraining.*

→ Open Google Colab to follow along

# Lab 3 — Quantization: The Math Behind Fewer Bits

## Uniform Affine Quantization

**Scale:**  $s = (x_{\max} - x_{\min}) / (2^b - 1)$

**Zero-point:**  $z = -\text{round}(x_{\min} / s)$

**Quantize:**  $q = \text{clamp}(\text{round}(x/s) + z, 0, 2^b - 1)$

**Dequantize:**  $\hat{x} = s \times (q - z)$

**Max error:**  $|x - \hat{x}| \leq s/2$

### Memory Savings by Precision

|                                                                                     |                                          |
|-------------------------------------------------------------------------------------|------------------------------------------|
|  | <b>FP32 (32 bits)</b><br>1.0× (baseline) |
|  | <b>FP16 (16 bits)</b><br>0.5× smaller    |
|  | <b>INT8 (8 bits)</b><br>0.25× smaller    |
|  | <b>INT4 (4 bits)</b><br>0.125× smaller   |

### Why FP16 is Nearly Lossless

FP16 = 1 sign bit + 5 exponent bits + 10 mantissa bits → dynamic range  $\sim 10^{-4}$  to  $10^4$ .  
Most neural network weights live in  $[-2, 2]$ , so FP16 rarely causes overflow or significant rounding error.  
Result:  $\sim 1.5$ – $2\times$  speedup with near-zero accuracy loss. Enable it with a single flag: `half=True`

## Lab 3 — FP16 vs INT8: When to Use Each

### FP16 (Half Precision)

#### ✓ Advantages:

- + Zero accuracy loss on modern GPUs
- + Enabled with one flag (half=True)
- + Works on all CUDA-capable hardware
- + ~1.5–2× faster on T4, A100, RTX

#### ✗ Limitations:

- No benefit on hardware without FP16 units
- Slight memory overhead vs INT8

**Recommendation:** Use FP16 always as a first optimization. It's free speed.

### INT8 (8-bit Integer)

#### ✓ Advantages:

- + 4× memory savings vs FP32
- + 2–4× faster on dedicated INT8 hardware
- + Ideal for Jetson Nano TensorRT
- + Enables running 4+ cameras on edge

#### ✗ Limitations:

- Requires calibration dataset
- ~1–2% mAP drop
- Needs TensorRT or ONNX runtime

**Recommendation:** Use INT8 for edge deployment (Jetson Nano) after FP16.

### ONNX Runtime

#### ✓ Advantages:

- + Hardware-agnostic inference
- + Works on CPU, GPU, edge
- + Easy export from PyTorch/YOLO
- + Enables cross-platform deployment

#### ✗ Limitations:

- Slightly slower than TensorRT
- Less tuned for specific hardware
- No training — inference only

**Recommendation:** Use ONNX as a step toward TensorRT on edge devices.



Lab 4

# Knowledge Distillation: Teacher → Student

---

*Objective: Train a small 'student' model to mimic a large 'teacher' model.  
Get near-teacher accuracy at student-level cost.*

→ [Open Google Colab to follow along](#)

## Why Does the Teacher Know More Than the Labels?

### Hard Labels (Ground Truth)

car: 1.0 | van: 0.0 | truck: 0.0 | bus: 0.0

Hard labels throw away information. They only say 'this IS a car' — not 'this looks kind of like a van too'.

### Soft Labels (Teacher Output — Dark Knowledge)

car: 0.80 | van: 0.15 | truck: 0.04 | bus: 0.01

Soft labels encode that a car is similar to a van! This 'dark knowledge' helps the student generalize far better than hard labels alone.

### Distillation Loss — Hinton et al. 2015

Temperature Scaling (softens teacher output):

$$p_i^T = \exp(z_i/T) / \sum_j \exp(z_j/T)$$

*Higher  $T \rightarrow$  softer distribution  $\rightarrow$  more dark knowledge*

### Combined KD Loss:

$$L_{KD} = (1-\alpha) \cdot L_{CE}(\text{hard}) + \alpha \cdot T^2 \cdot L_{KL}(\text{teacher} || \text{student})$$

$\alpha$  — Distillation weight (0.5–0.9 typical)

$T$  — Temperature (4 is common; try 2–8)

$L_{CE}$  — Cross-entropy with ground truth labels

$L_{KL}$  — KL divergence: teacher vs student soft outputs

$T^2$  — Gradient scale compensation for temperature

# Lab 4 — Why Distillation Solves the Multi-Camera Problem

If you have  $N$  cameras, you need  $N$  inference passes per second.

| # Cameras | Teacher (YOLOv8s ~120 FPS) | Student (YOLOv8n ~300 FPS) |
|-----------|----------------------------|----------------------------|
| 1         | 120 FPS → ✓                | 300 FPS → ✓                |
| 2         | 60 FPS → ✓                 | 150 FPS → ✓                |
| 4         | 30 FPS → ✓                 | 75 FPS → ✓                 |
| 8         | 15 FPS → ✗                 | 37 FPS → ✓                 |
| 16        | 7 FPS → ✗                  | 18 FPS → ✗                 |

Real-time threshold: 25 FPS per camera

## The Distillation Win

- YOLOv8n (student) trained with KD achieves ~92% of YOLOv8s (teacher) mAP
- But runs 2.5× faster — allowing 4 cameras where teacher could only handle 2
- This is the power of distillation: you don't lose the teacher's knowledge, you compress it.**

Lab 5

# Multi-Scale Heads & Architecture Design

---

*Objective: Understand why multi-scale detection heads matter for tiny drone objects.  
Design lightweight heads using Depthwise-Separable convolutions.*

→ Open Google Colab to follow along

## Why Do We Need Multiple Detection Scales?

A pedestrian at drone altitude may be only 8×8 pixels. A single detection head tuned for large objects will miss it entirely.

| P3 (Stride 8)                                                                                                                                                                                    | P4 (Stride 16)                                                                                                                                                                                  | P5 (Stride 32)                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Grid: 52×52</b><br><br>Anchor points: 2,704<br><br><b>Best for: Tiny objects</b><br><b>&lt; 16 px</b><br><br>Equation: $\text{Grid} = \lfloor 416 / 8 \rfloor \times \lfloor 416 / 8 \rfloor$ | <b>Grid: 26×26</b><br><br>Anchor points: 676<br><br><b>Best for: Small objects</b><br><b>16–64 px</b><br><br>Equation: $\text{Grid} = \lfloor 416 / 16 \rfloor \times \lfloor 416 / 16 \rfloor$ | <b>Grid: 13×13</b><br><br>Anchor points: 169<br><br><b>Best for: Large objects</b><br><b>&gt; 64 px</b><br><br>Equation: $\text{Grid} = \lfloor 416 / 32 \rfloor \times \lfloor 416 / 32 \rfloor$ |

## YOLOv8 — Anchor-Free Detection (DFL)

YOLOv8 uses Distribution Focal Loss (DFL):  $\hat{b} = \sum_i \text{softmax}(z_i) \times i$   
The model learns a distribution over offsets rather than fixed anchor shapes → better for diverse small objects in drone footage.

## Depthwise-Separable Convolution — The Key Technique

### Standard Conv2D FLOPs:

$$\text{FLOPs} = C_{\text{in}} \times k^2 \times C_{\text{out}} \times H \times W$$

### Depthwise-Separable Conv FLOPs:

$$\begin{aligned} \text{FLOPs}_{\text{DW}} &= (C_{\text{in}} \times k^2 \times H \times W) \\ &+ (C_{\text{in}} \times C_{\text{out}} \times H \times W) \end{aligned}$$

### FLOPs Reduction Factor:

$$\text{Reduction} \approx (1/C_{\text{out}}) + (1/k^2)$$

For  $k=3$ ,  $C_{\text{out}}=256$ : reduction  $\approx 8\text{--}9\times$  fewer FLOPs!

### How DW-Sep Conv Works:

#### Step 1 — Depthwise Conv:

Apply one  $k \times k$  filter per input channel independently. This captures spatial features without mixing channels.

#### Step 2 — Pointwise Conv ( $1 \times 1$ ):

Apply  $1 \times 1$  convolution to mix channels and project to the desired output dimension.

#### Why it works:

Separating spatial filtering from channel mixing reduces redundant computation while preserving representational capacity.

For VisDrone: A DW-Sep head on P3 (stride 8) gives  $\sim 85\text{--}90\%$  FLOPs reduction in the detection head alone. The P3 head is critical — tiny pedestrians only appear here!

Lab 6

# Multi-Camera Scalability & Edge Deployment

---

*Objective: Export optimized models to ONNX, simulate multi-camera throughput, and understand Jetson Nano deployment.*

→ Open Google Colab to follow along



## How do you serve 4 cameras simultaneously on one GPU?

The answer: batch inference. Group N camera frames into a single forward pass.

### Why Batching Increases Throughput

- GPU computation is parallelized across the batch dimension
- Amortize fixed overhead (memory transfer, kernel launch) across N images
- $\text{Throughput}(\text{bs}=4) \approx 2\text{--}3\times \text{Throughput}(\text{bs}=1)$  for same model
- **BUT: latency per image increases — trade-off for live streams**

| Batch Size | Latency / batch | FPS (per img) | Total throughput |
|------------|-----------------|---------------|------------------|
| bs=1       | 8 ms            | 125           | 125 img/s        |
| bs=2       | 10 ms           | 100           | 200 img/s        |
| bs=4       | 15 ms           | 67            | 267 img/s        |
| bs=8       | 25 ms           | 40            | 320 img/s        |

Example: YOLOv8n FP16 on T4 GPU (estimated)

Required throughput for 4 cameras at 25 FPS = 100 images/sec.  
With YOLOv8n FP16 + batch size 4: throughput  $\approx 267$  img/s  $\rightarrow$  Meets the requirement with headroom!

## Steps to Deploy Your Optimized Model on Jetson Nano

### Step 1: Export to ONNX (in Colab)

```
model.export(format='onnx', imgsz=416, simplify=True)  
Simplify=True reduces graph complexity for edge runtime.
```

### Step 2: Convert to TensorRT (on Jetson)

```
trtexec --onnx=model.onnx \  
--saveEngine=model.engine \  
--fp16  
# Add --int8 for maximum speed
```

### Step 3: Run Inference on Jetson

```
from ultralytics import YOLO  
m = YOLO('model.engine')  
results = m.predict(source=0, stream=True) # webcam
```

### Step 4: Multi-Camera Streaming

```
sources = ['rtsp://cam1', 'rtsp://cam2',  
           'rtsp://cam3', 'rtsp://cam4']  
results = m.predict(source=sources, stream=True)
```

### Expected Jetson Nano Performance:

Single camera: ~20-25 FPS | Dual camera: ~12-15 FPS/stream | 4 cameras: ~6-8 FPS/stream (YOLOv8n INT8)

# All Techniques Side-by-Side — Expected Results Dashboard

Every optimization we did — measured and compared

| Technique              | FLOPs Reduction | Latency Improvement | mAP Impact | Effort |
|------------------------|-----------------|---------------------|------------|--------|
| Baseline (FP32)        | 0%              | 1×                  | 0%         | ★      |
| FP16 Quantization      | 0%              | ~1.5–2×             | ~0%        | ★      |
| INT8 (TensorRT)        | 0%              | ~2–4×               | ~1–2%      | ★★     |
| Structured Pruning     | 20–40%          | ~1.2–2×             | ~2–5%      | ★★★    |
| Knowledge Distillation | 60–75%          | ~2–3×               | ~5–8%      | ★★★    |
| DW-Sep Heads           | 85–90% (head)   | ~1.3×               | ~1–3%      | ★★     |
| Image Resize ↓         | Quadratic       | Quadratic           | ~3–10%     | ★      |

## The Pareto Frontier Principle

No single optimization wins everywhere. The art of real-world AI engineering is finding the optimal point on the mAP ↔ Speed ↔ Memory frontier, given your specific constraints (cameras, hardware, acceptable accuracy drop).

# Final Hackathon Challenge — Design for 4 Cameras at 25 FPS

Mission: Design a system to run 4 cameras at 25 FPS each using one GPU (T4 or Jetson Nano), keeping mAP@50  $\geq$  80% of baseline.

| Option                | Est. Total FPS | Meets 100 FPS? | mAP $\geq$ 80%? |
|-----------------------|----------------|----------------|-----------------|
| YOLOv8s FP32          | ~120           | ✗              | ✓               |
| YOLOv8s FP16          | ~200           | ✓              | ✓               |
| YOLOv8n FP16          | ~300           | ✓              | ✓ (~87%)        |
| YOLOv8n FP16 + KD     | ~300+          | ✓              | ✓ (~92%)        |
| YOLOv8n INT8 (Jetson) | ~25/cam        | ✓              | ✓ (~89%)        |

## Best Recommendation: YOLOv8n FP16 + Knowledge Distillation

- Highest FPS while keeping mAP within 8% of baseline
- With batch inference (bs=4): throughput increases ~2× further → exceeds 600 img/s
- For Jetson Nano: YOLOv8n INT8 gives ~20-25 FPS per camera — adequate for 1-2 cameras, scale to 4 with a stronger board

# Beyond This Workshop — Future Directions & Career Paths

*You've mastered the fundamentals. Here's where to go next.*

## Neural Architecture Search (NAS)

Automate architecture design. Tools: AutoML, DARTS, Once-for-All. Use NAS to find the best model for your hardware constraint automatically.

## Quantization-Aware Training (QAT)

Train with simulated quantization — more accurate than post-training INT8. Use it when PTQ accuracy drop is unacceptable.

## Vision Transformers (ViT) for Detection

DETR, RT-DETR, Co-DETR. Attention-based detection — different optimization strategies apply (token pruning, attention distillation).

## Streaming & Pipeline Optimization

DeepStream (NVIDIA), GStreamer pipelines. Multi-camera systems need software plumbing beyond just the model — scheduling, buffering, decoding.

## Domain Adaptation

Your model trained on VisDrone may fail on a different city or weather. Learn UDA (Unsupervised Domain Adaptation) to close this gap.

## Dataset & Annotation at Scale

Better data > bigger model. Active learning, semi-supervised annotation, synthetic data from CARLA/AirSim for drone scenarios.

# Key Takeaways — What You Can Do After Today

## FP16 is Always On

Half precision gives  $\sim 1.5\text{--}2\times$  speedup with near-zero accuracy cost. Always enable it. INT8 + TensorRT for edge deployment.

## Pruning Removes Redundancy

BN  $\gamma$  values reveal which channels are idle. Structured pruning removes them entirely — real FLOPs savings. Fine-tune to recover accuracy.

## Multi-Scale Heads Detect Tiny Objects

Drone footage needs the P3 (stride 8) head for tiny objects. DW-Sep convolutions reduce head FLOPs by 8–9 $\times$  without accuracy sacrifice.

## Deploy with Batch + ONNX + TensorRT

Batch inference is the practical multi-camera solution. Export ONNX  $\rightarrow$  TensorRT on Jetson for hardware-optimized edge inference.



*Fast models don't just run faster —  
they enable new applications that  
slow models can't even attempt.*



---

**Congratulations on completing the SAIE 2026 Workshop!**

You are now equipped to:

**Profile · Prune · Quantize · Distill · Design Heads · Deploy to Edge**

---

Teerapong Panboonyuen, Ph.D. (P'Kao)

Get to know me at <https://kaopanboonyuen.github.io/>  
*Super AI Engineer Thailand — Powered by NECTEC & AIAT*